

Copilot Meta-Prompting

사내 Copilot을 위한 메타 프롬프팅 Extension

팀

PaPaChu

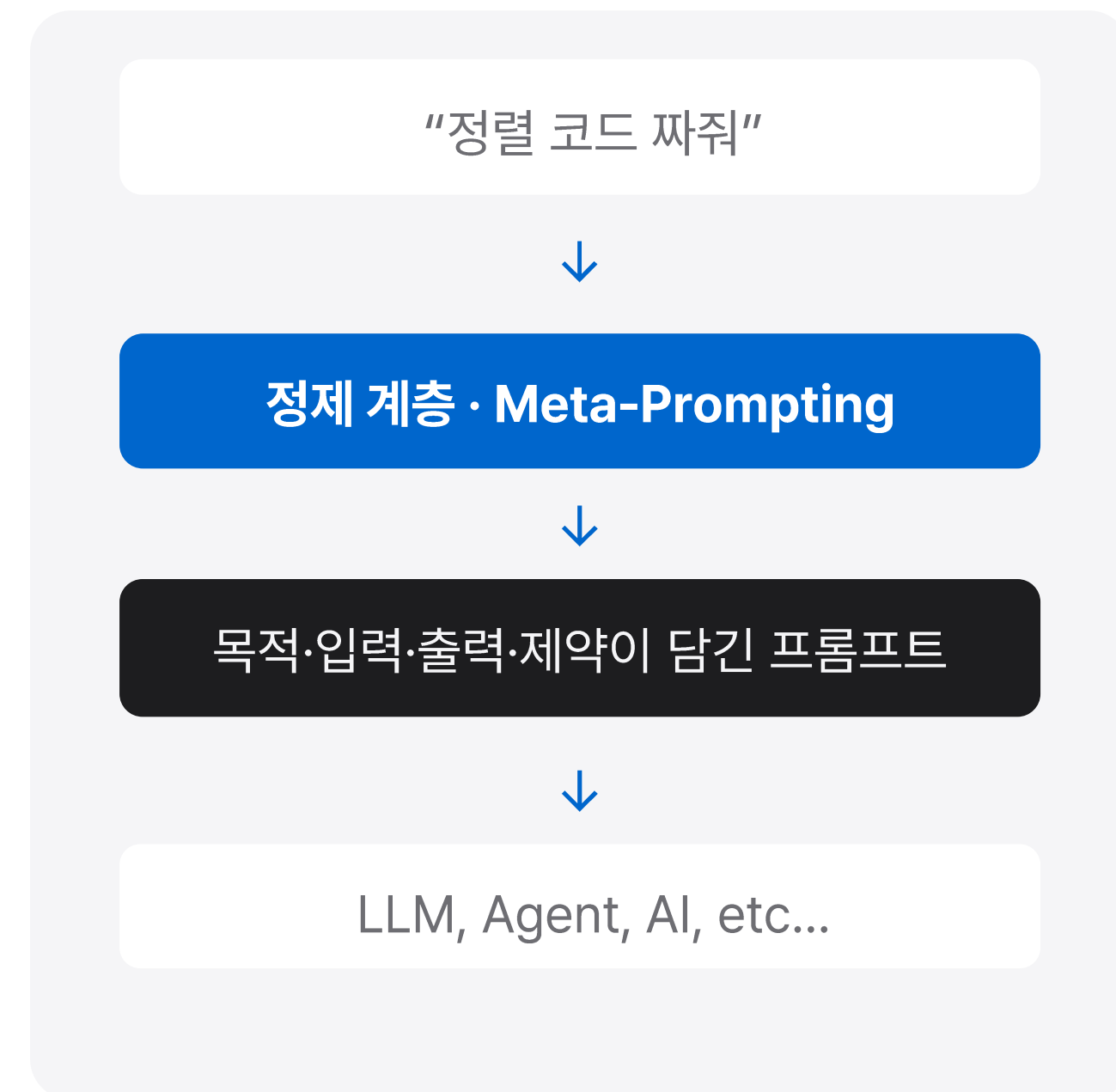
팀원

김태용 · 김민정 · 박혜림 · 홍창표

메타 프롬프팅이란

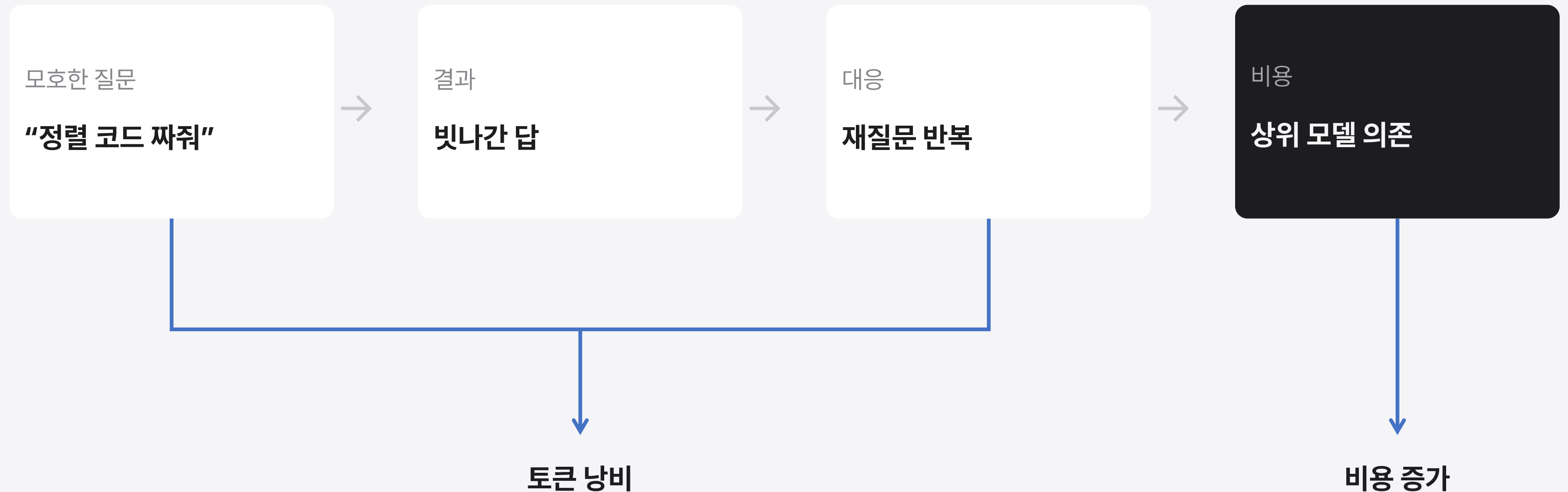
- 메타 프롬프팅(Meta-Prompting)은 AI에게 좋은 답변을 끌어내기 위한 '프롬프트 설계 자체를 AI에게 위임하는 기법'

- 사용자와 AI 사이의 정제 계층
- 모호한 입력 → 구조적 프롬프트로 재작성
- 목적 · 맥락 · 제약을 명시



왜 필요한가

모호한 질문이 만드는 악순환

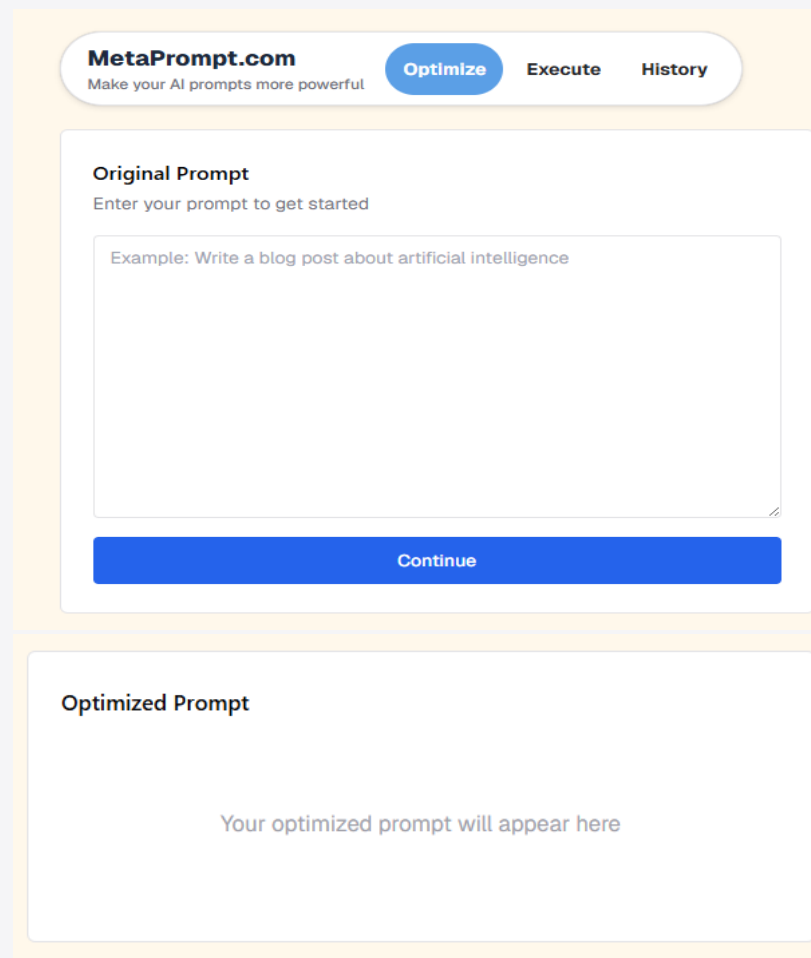


매 요청마다 사람이 직접 프롬프트를 작성 하는 것은 노동 집약적인 작업
최대한 One-shot에 작업을 할 수 있게 해보자!

사내에서 기존 방식의 한계

외부 도구 왕복

Copilot과 따로 노는 도구



The screenshot shows the MetaPrompt.com website. At the top, there's a header with the logo and navigation buttons: 'Optimize', 'Execute', and 'History'. Below this, there's a section titled 'Original Prompt' with the instruction 'Enter your prompt to get started'. A text box contains an example: 'Example: Write a blog post about artificial intelligence'. Below the text box is a blue 'Continue' button. Further down, there's a section titled 'Optimized Prompt' with the placeholder text 'Your optimized prompt will appear here'.

복사 붙여 넣기의 반복

정보 유출

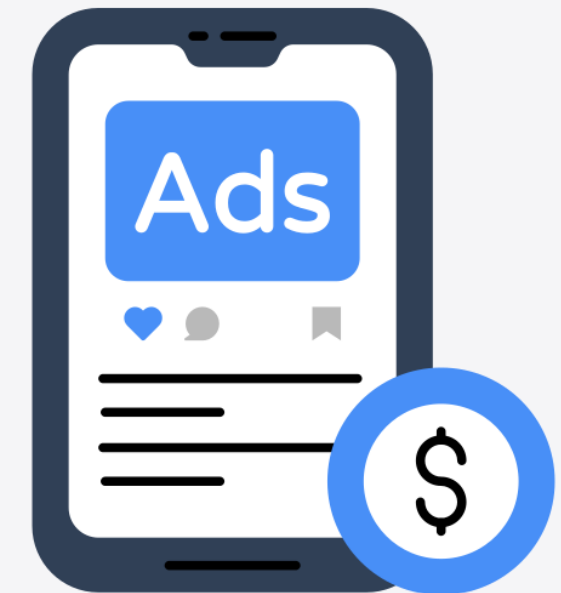
사내 코드·자료 외부 전송



보안 통제 밖으로 데이터 이동
사내에서 접근 자체가 막혀 있는 경우

비용 문제

외부 API KEY를 요구

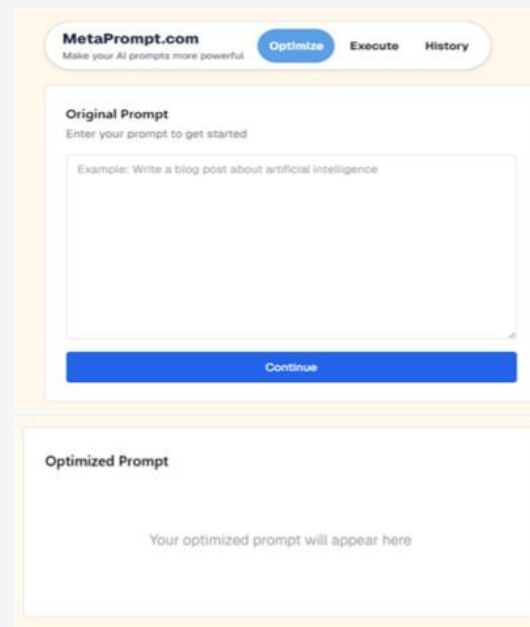


혹은 서비스 자체가 유료

프로젝트 목적

외부 도구 왕복

Copilot과 따로 노는 도구



복사 붙여넣기의 반복

정보 유출

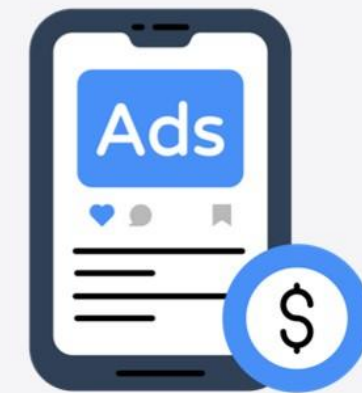
사내 코드·자료 외부 전송



보안 통제 밖으로 데이터 이동
사내에서 접근 자체가 막혀 있는 경우

비용 문제

외부 API KEY를 요구



혹은 서비스 자체가 유료

01

기존 Copilot 사용법 그대로

Copilot Chat + @refine · 새로 배울 것 없음

02

승인 후 전송

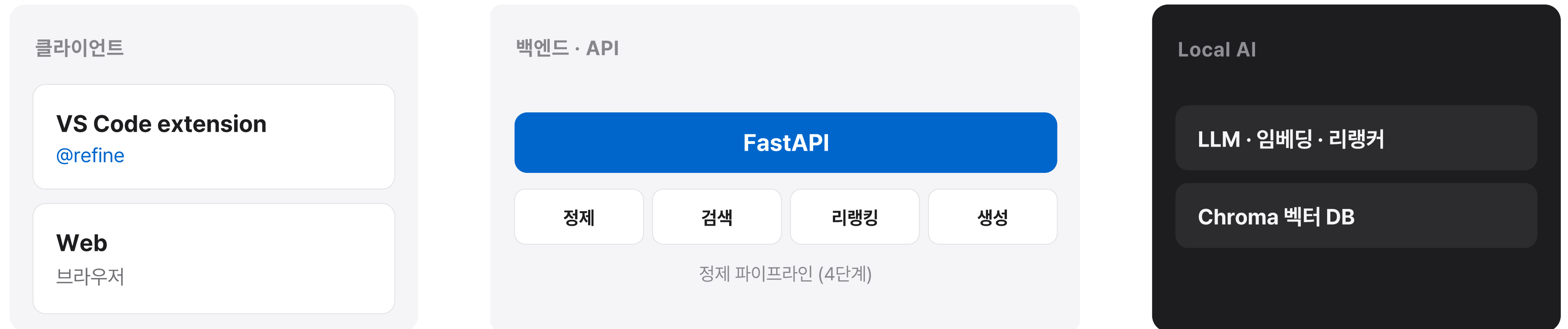
정제본 미리 보기 · 사용자 통제 및 수정

03

Local AI 사용

사내 처리 · 외부 전송 없음 · 서비스 비용 최소화

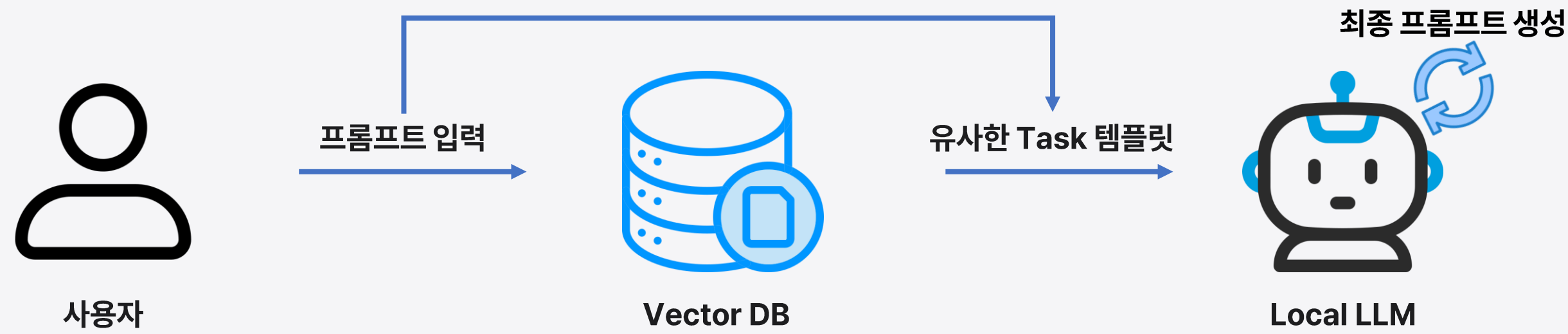
전체 아키텍처



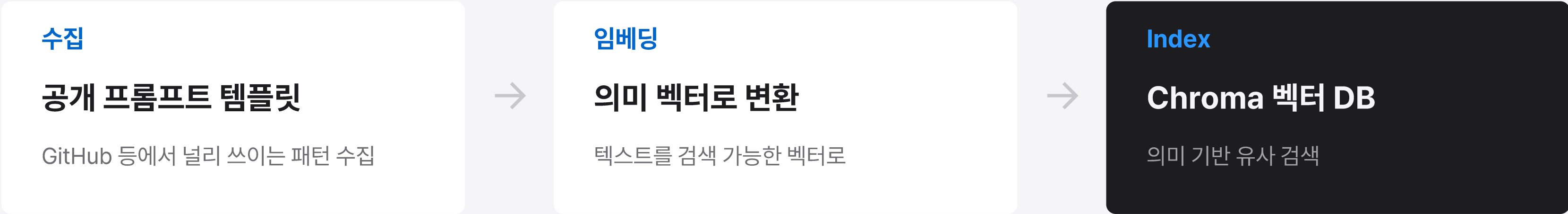
Client, backend, AI 3가지 모듈로 구성된 아키텍처

템플릿과 벡터 DB

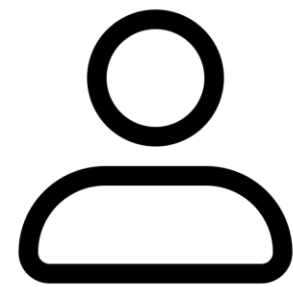
미리 색인해 둔 프롬프트 템플릿을 검색에 활용



Local AI의 성능을 극복하기 위한 RAG 기반 접근법 사용



정제 파이프라인



사용자



1

정제

사용자 입력 → 명확한 프롬프트로 재작성
검색 쿼리를 생성하는 단계
LLM 사용

2

검색

벡터 DB에서 유사 템플릿 후보 검색
TOP-20 추출
Vector DB 사용

3

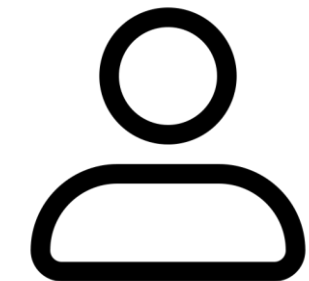
리랭킹

검색된 20개의 후보 중에 top-3 선정
노이즈 감소, 제한 된 context length 극복
Reranking Model 사용

4

최종 프롬프트 생성

최종 top-3 템플릿 + 사용자 원본 프롬프트
→ 최종 프롬프트 생성
LLM 사용



사용자

실제 예시

원본

“레이트 리미터 만들어줘”



정제본

역할	대규모 트래픽 백엔드·분산 시스템 아키텍트 — 동시성·부하 분산·안정성 전문
임무	프로덕션급 Rate Limiter 설계·구현 — 단순 코드를 넘어 견고한 아키텍처 제안
필수 포함	① 알고리즘 비교 — Token Bucket · Sliding Window · Fixed Window (버스트·메모리 트레이드오프) ② 동시성 안전 구현 — Python/Go, Redis 분산으로 단일 인스턴스 한계 극복. 한국어 주석 첨부 ③ Race Condition 방지 — Lock 메커니즘 · Atomic 연산 ④ 아키텍처 — 배치 위치(Gateway vs App) · 파라미터(window·max) · 예외(HTTP 상태코드)
출력 형식	알고리즘 선택 근거 · 아키텍처 다이어그램 · 구현 코드 블록 · 동시성·확장성 요약

결과 · VS Code 확장(시연)

VSIX 설치

VS Code에서 설치가능한 설치파일 배포

@refine 호출

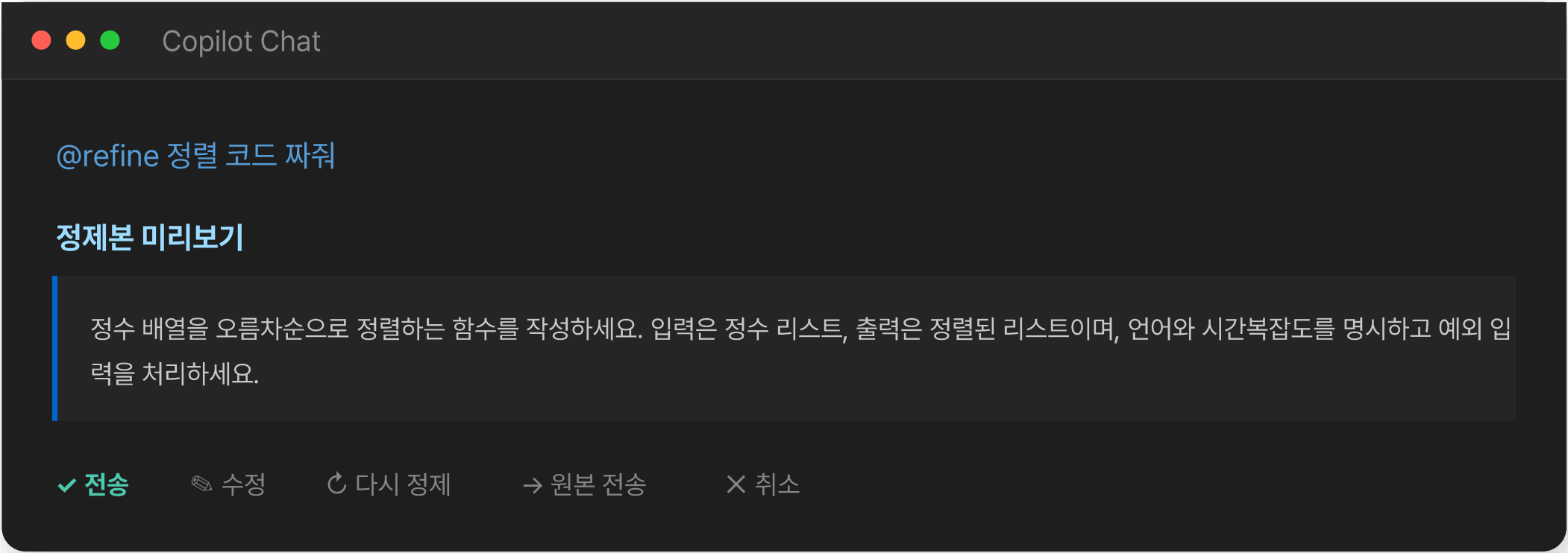
Copilot Chat에서 사용자가 Enter 입력시 명령을
가로채어 메타 프롬프팅 서버로 전송

승인 게이트

전송 · 수정 · 다시 정제 · 원본 전송 · 취소

사용자 통제

승인 해야만 Copilot의 원본 피커 모델로 전송



결과 · 웹 데모(시연)

설치 없이

브라우저에서 바로 사용

4단계 시각화

정제 → 검색 → 리랭킹 → 생성

투명성

중간 산출물·최종 프롬프트 확인

메타 프롬프팅 파이프라인

예) AI 자유 주제로 5분 발표 슬라이드를 구성해줘

1 정제
재작성

2 검색
유사 템플릿

3 리랭킹
재정렬

4 생성
최종 프롬프트

장점

보안

로컬 AI 처리 · 사내 자료 외부 유출 없음

비용

저비용 모델로도 고품질 · 상위 모델 의존↓

익숙함

기존 Copilot 그대로 · @refine 한 단어

한계점

서버 필요

로컬 AI 구동을 위한 GPU 자원 필요

플래그십 모델에서 효과 미미

클로드 Opus와 같은 모델에서
메타 프롬프팅 효과 차이 미미

단순한 파이프라인

멀티턴 메타 프롬프팅과 같은 고도화 된
에이전틱 기능의 부재

마무리

감사합니다.

Copilot Meta-Prompting